

CST 8506 – Machine Learning

Final Project – Group 8

Leaf Species Classification Using Machine Learning

Applying CRISP-DM to the UCI Leaf Dataset

Hye Ran Yoo
041145212

Peng Wang
041107730

Yinyuan Chen
041146649

Apr 3, 2026 | Algonquin College

1. Introduction & Project Overview
2. Business Understanding
3. Data Understanding – Feature & Image Datasets
4. **Classification (Hye Ran Yoo)**
 - Data Preparation, Modelling, Evaluation
5. **Clustering (Peng Wang)**
 - Data Preparation, Modelling, Evaluation
6. **Outlier Detection (Yinyuan Chen)**
 - Data Preparation, Modelling, Evaluation
7. Discussion of Results
8. Conclusion & Future Work
9. Q & A

Presenter Assignments

Hye Ran Yoo : Total Slides 11

Data Understanding (Slides 6-7)

Classification (Slides 13–21)

Peng Wang : Total Slides 10

Explore Data, Data Quality (Slides 8-11)

Clustering (Slides 23–27)

Conclusion (Slides 36)

Yinyuan Chen : Total Slides 10

Intro, Business Understanding (Slides 2-5)

Outlier Detection (Slides 29–32)

Discussion (Slides 34–35)

1. Introduction

Presenter: Yinyuan Chen

- Plant species identification is critical in botany, ecology, agriculture, and environmental conservation
- Traditional approaches rely on manual inspection by trained experts
→ time-consuming and prone to subjective errors
- This project applies machine learning methods to the UCI Leaf Dataset
 - 340 leaf specimens from 30 plant species
 - 14 numerical features extracted from digital images
 - Additional RGB and BW image datasets
- Three core tasks:
 - 1) Classification (Peng Wang)
 - 2) Clustering (Hye Ran Yoo)
 - 3) Outlier Detection (Yinyuan Chen)
- Framework: CRISP-DM (Cross-Industry Standard Process for Data Mining)

```
# Descriptive statistics
feature_cols = column_names[2:] # 14 numerical features

print("=" * 60)
print("DESCRIPTIVE STATISTICS")
print("=" * 60)
print(f"Number of features: {len(feature_cols)}")
print(f"Total instances: {df.shape[0]}")
print(f"Number of species: {df['Class'].nunique()}")
print()
df[feature_cols].describe().round(4)
```

```
# Class distribution
class_dist = df['Class'].value_counts().sort_index()
print(f"Classes: {len(class_dist)}, Min per class: {class_dist.min()}, Max per class: {class_dist.max()}")
print(f"Class labels: {sorted(df['Class'].unique())}")
print()

# Non-consecutive labels
missing_labels = [l for l in range(1, 37) if l not in df['Class'].unique()]
print(f"Missing class labels: {missing_labels}")
print("These correspond to species only in the RGB folder, not in the feature dataset.")
print()
print(class_dist.to_frame('Count'))
```

2. Business Understanding – Project Plan

Presenter: Yinyuan Chen

Workload Distribution

Task / Subtask	Student Name
Introduction	All Members-Peng Wang
Business Understanding	All Members-Yinyuan Chen
Data Understanding	All Members-Hye Ran Yoo
Classification – Prep, Model, Eval	Hye Ran Yoo
Clustering – Prep, Model, Eval	Peng Wang
Outlier Detection – Prep, Model, Eval	Yinyuan Chen
Discussion of Results	All Members
Conclusion	All Members

Project Timeline

Phase	Deadline
Part 1: Business + Data + Prep	March 27, 2026
Part 2: Modelling	Mar 28 – Apr 1
Report Finalization	April 2, 2026
Presentation	April 8, 2026

2. Business Understanding – Objectives & Situation

Presenter: Yinyuan Chen

Business Objectives:

- Develop a reliable system for automatic plant species recognition
- Evaluate how well different ML techniques distinguish 30 plant species
- Compare traditional ML on features vs. deep learning on images

Situation Assessment:

- UCI Leaf Dataset – publicly available, CC BY 4.0 license
- Small dataset: 340 instances across 30 classes (8–16 per species)
- Image dataset: 443 RGB images (40 species), 340 BW images (30 species)
- Tools: Python, scikit-learn, PyTorch, Power BI

Data Mining Goals:

- Classification: Naïve Bayes, SVM, MLP, Stacking, CNN
- Clustering: K-Means, DBSCAN, GMM (EM)
- Outlier Detection: LOF, Isolation Forest, One-Class SVM

Success Metrics: Accuracy, Precision, Recall, F1, ARI, NMI, Silhouette

3. Data Understanding – Feature Dataset

Presenter: Hye Ran Yoo

- Source: UCI Machine Learning Repository (Silva & Marçal, 2013)
- **leaf.csv: 340 instances, 16 columns**
 - Column 1: Species class label (**30** unique, non-consecutive: 1–15, 22–36)
 - Column 2: Specimen number
 - Columns 3–16: **14** numerical features

- **Features divided into two groups:**

Shape: Eccentricity, Aspect Ratio, Elongation, Solidity,

Stochastic Convexity, Isoperimetric Factor,

Max Indentation Depth, Lobedness

Texture: Avg Intensity, Avg Contrast, Smoothness,

Third Moment, Uniformity, Entropy

```
# Paths are defined in the Google Drive setup cell above

# Load feature dataset
column_names = [
    'Class', 'Specimen_Number', 'Eccentricity', 'Aspect_Ratio',
    'Elongation', 'Solidity', 'Stochastic_Convexity', 'Isoperimetric_Factor',
    'Max_Indentation_Depth', 'Lobedness', 'Avg_Intensity', 'Avg_Contrast',
    'Smoothness', 'Third_Moment', 'Uniformity', 'Entropy'
]

df = pd.read_csv(CSV_PATH, header=None, names=column_names)
print(f"Feature dataset loaded: {df.shape[0]} rows, {df.shape[1]} columns")
df.head()
```

Feature dataset loaded: 340 rows, 16 columns

	Class	Specimen_Number	Eccentricity	Aspect_Ratio	Elongation	Solidity	Stochastic_Convexity	Isoperimetric_Factor	Max_Indentation_Depth	Lobedness	Avg_Intensity	Avg_Contrast	Smoothness	Third_Moment	Uniformity	Entropy
0	1	1	0.72694	1.4742	0.32396	0.98535	1.00000	0.83592	0.004657	0.003947	0.047790	0.127950	0.016108	0.005232	0.000275	1.17560
1	1	2	0.74173	1.5257	0.36116	0.98152	0.99825	0.79867	0.005242	0.005002	0.024160	0.090476	0.008119	0.002708	0.000075	0.69659
2	1	3	0.76722	1.5725	0.38998	0.97755	1.00000	0.80812	0.007457	0.010121	0.011897	0.057445	0.003289	0.000921	0.000038	0.44348
3	1	4	0.73797	1.4597	0.35376	0.97566	1.00000	0.81697	0.006877	0.008607	0.015950	0.065491	0.004271	0.001154	0.000066	0.58785
4	1	5	0.82301	1.7707	0.44462	0.97698	1.00000	0.75493	0.007428	0.010042	0.007938	0.045339	0.002051	0.000560	0.000024	0.34214

Classes: 30, Min per class: 8, Max per class: 16

Class labels: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(10), np.int64(11), np.int64(12), np.int64(13), np.int64(14), np.int64(15), np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20), np.int64(21), np.int64(22), np.int64(23), np.int64(24), np.int64(25), np.int64(26), np.int64(27), np.int64(28), np.int64(29), np.int64(30)]

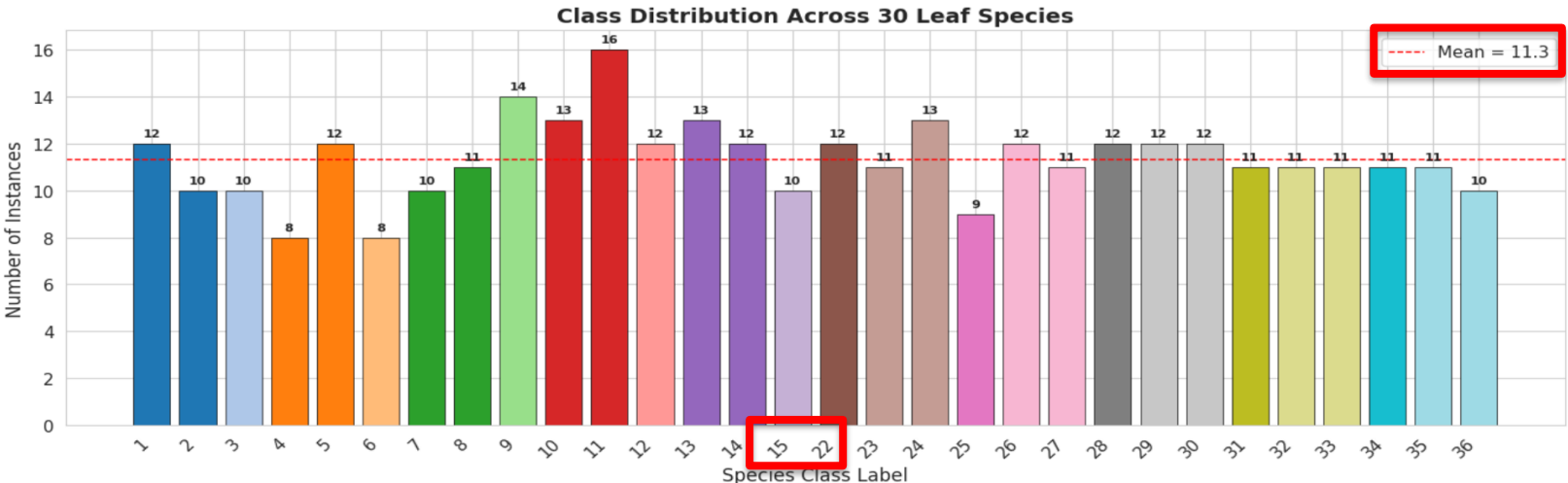
Missing class labels: [16, 17, 18, 19, 20, 21]

These correspond to species only in the RGB folder, not in the feature dataset.

3. Data Understanding – Image Dataset

- **Data Sources & Usages:**
 - CSV (14 features): Main input dataset (PCA, LDA, SVM, Stacking)
 - RGB Images (443 imgs): Input for CNN feature extraction
 - BW Images (340 imgs): Used for analyzing leaf contours and shapes
- **Two Image Types:** RGB (443 imgs) & BW (340 imgs)
- **Dataset Size Issue:** 340 total images is **too small** for Deep Learning (CNN) → Explains why **SVM** performed better
- **Class Mismatch:** 40 species (RGB) vs. 30 species → **Filtering** required
- **Dimension Inconsistency:**
 - Found mixed image orientations (720x960 & 960x720) during Quality Check → **Forced resize to 128x128px** for CNN input
- **Missing Classes:** IDs **16~21** are absent in the main feature data
- **Class Distribution:** Average 11.3 imgs per class → Relatively balanced

Property	RGB Images	BW Images
Total images	443	340
Species (folders)	40	30
Dimensions	720 × 960 px	720 × 960 px
Colour mode	RGB (3-channel)	Binary (1-bit)
File format	JPG	JPG



3. Explore Data – Feature Distributions

Presenter: Peng Wang

- **Visualizing Outliers: For Outlier Detection**

Visually proves how many extreme, abnormal data points (black circles) exist in our dataset.

- **Standardized Scale:**

All features scaled to Mean=0 for easy comparison.

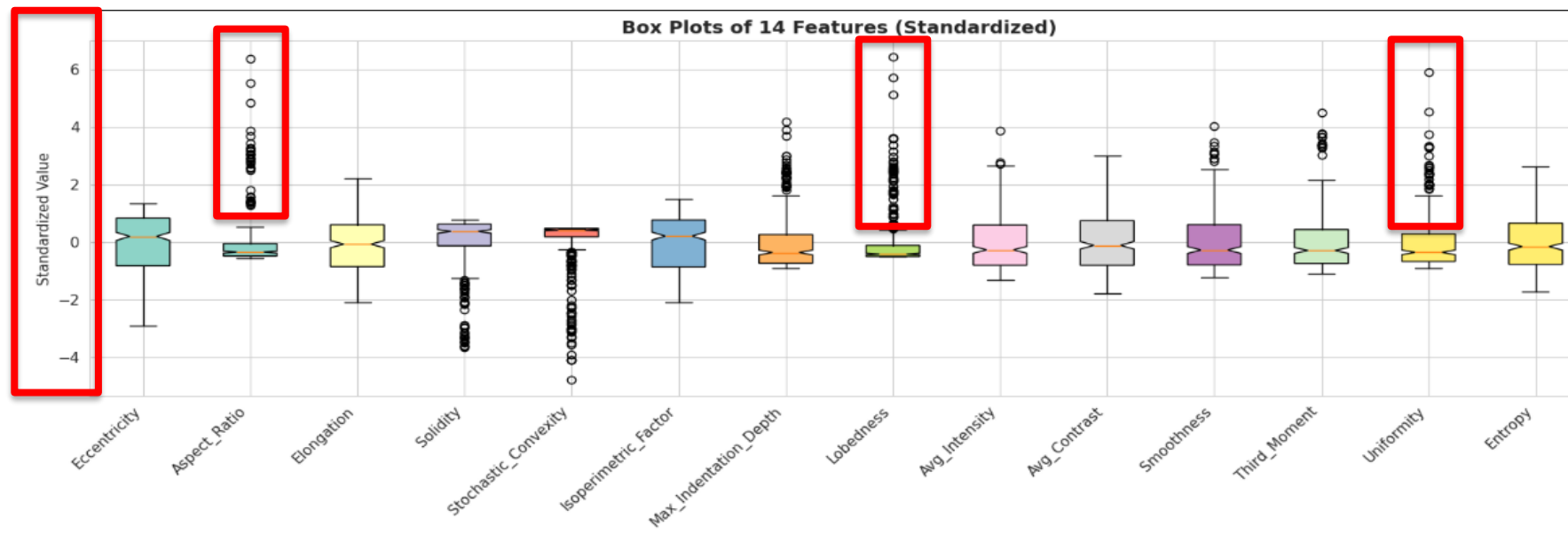
- **Key Point(Block Circles) :**

Bad : Confuses AI → Needs **Outlier Detection**.

Golden Hint: Easily find uniquely shaped plants!

- **Our Strategy:**

Instead of dropping outliers and losing rare leaf data → used **PCA & LDA** to reduce noise (while keeping valuable hints)



3. Explore Data – Feature Distributions (Histograms)

Presenter: Peng Wang

- **Messy Distributions :**

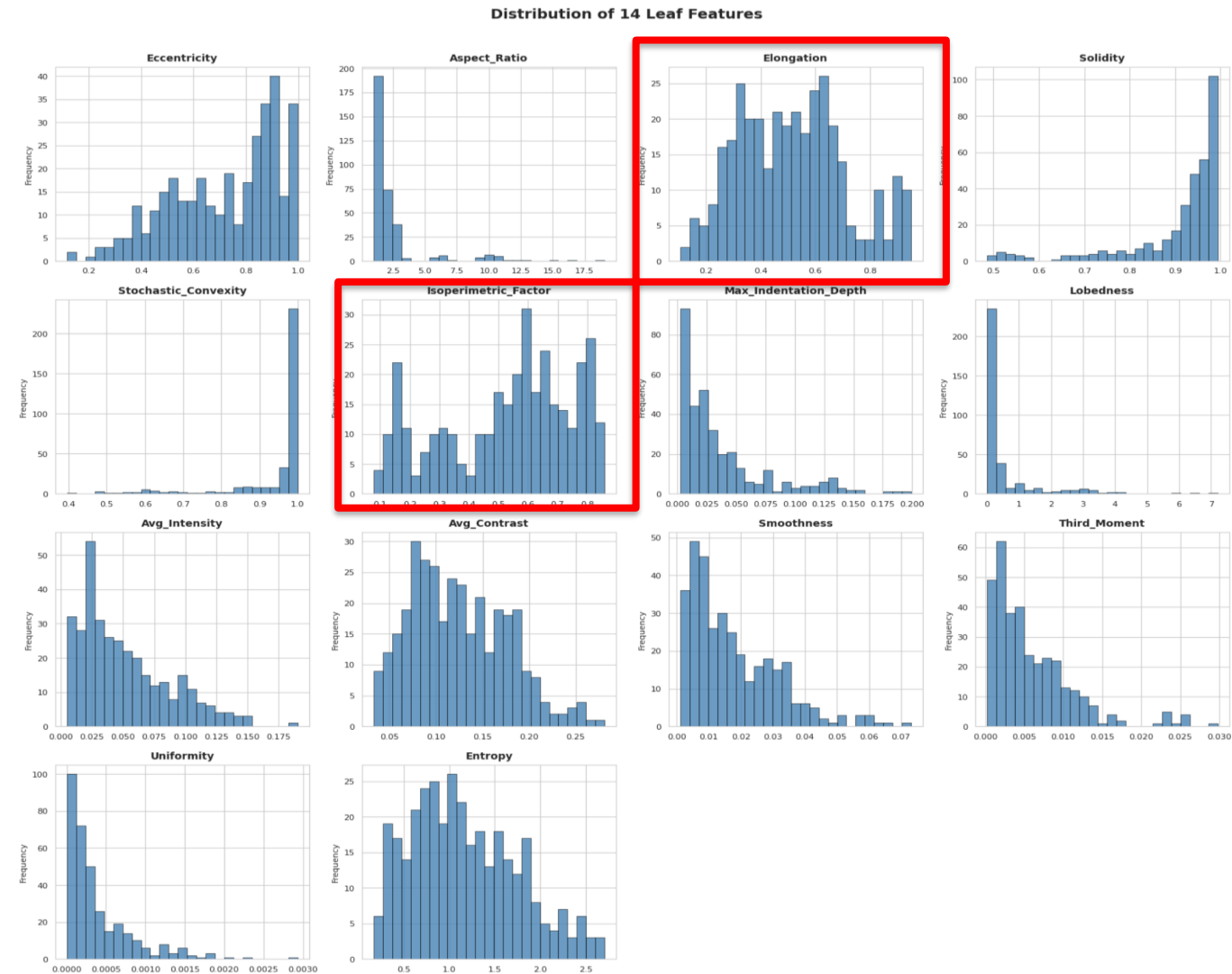
Most features are heavily skewed to one side (e.g., Lobedness, Uniformity).

- **Good Features (Red Boxes):**

'Elongation' is widely spread out, giving the AI clear hints to classify species.

- **Conclusion:**

Features have completely different shapes and scales
→ **Standardization is strictly required!**



3. Data Quality Verification

Presenter: Peng Wang

- **Feature Dataset Quality:**

- ✓ No missing values in all 340 rows × 16 columns
- ✓ No duplicate rows
- ✓ All features are float type; labels & specimen # are integers
- ✓ All values within plausible ranges
- ✓ 30 class labels match 30 BW image species folders

- **Image Dataset Quality:**

- ✓ All images open without errors
- ✓ Consistent 720 × 960 pixel dimensions
- ✓ BW folder image count matches feature dataset per species

- **Note: RGB folder has 10 extra species (16–21, 37–40)**

not present in the feature dataset

→ Dataset is clean and well-structured

→ No special treatment needed beyond standard preparation

```
# Image dataset quality checks
print("=" * 60)
print("IMAGE DATASET QUALITY CHECKS")
print("=" * 60)

bw_errors = 0
rgb_errors = 0
bw_dims = set()
rgb_dims = set()

for sp in bw_species:
    sp_path = os.path.join(BW_DIR, sp)
    for f in os.listdir(sp_path):
        try:
            img = Image.open(os.path.join(sp_path, f))
            bw_dims.add(img.size)
        except:
            bw_errors += 1

for sp in rgb_species:
    sp_path = os.path.join(RGB_DIR, sp)
    for f in os.listdir(sp_path):
        try:
            img = Image.open(os.path.join(sp_path, f))
            rgb_dims.add(img.size)
        except:
            rgb_errors += 1

print(f"BW corrupted files: {bw_errors}")
print(f"RGB corrupted files: {rgb_errors}")
print(f"BW dimensions: {bw_dims} (consistent: {len(bw_dims) == 1})")
print(f"RGB dimensions: {rgb_dims} (consistent: {len(rgb_dims) == 1})")

# Verify BW folder counts match feature data
match_count = 0
for sp in bw_species:
    class_num = int(sp.split('.')[0].strip())
    img_cnt = len(os.listdir(os.path.join(BW_DIR, sp)))
    feat_cnt = len(df[df['Class'] == class_num])
    if img_cnt == feat_cnt:
        match_count += 1

print(f"\nBW folder counts match feature data: {match_count}/{len(bw_species)}")
print("\nConclusion: Dataset is clean with no significant quality issues.")
```

3. Data Quality Verification

Presenter: Peng Wang

```
# Feature dataset quality checks
print("=" * 60)
print("FEATURE DATASET QUALITY CHECKS")
print("=" * 60)

missing = df.isnull().sum()
print(f"Missing values: {missing.sum()} (none found)")

dup_count = df.duplicated().sum()
print(f"Duplicate rows: {dup_count}")

print("\nData types:")
for col in df.columns:
    print(f" {col}: {df[col].dtype}")

print("\nFeature value ranges:")
for col in feature_cols:
    print(f" {col}: [{df[col].min():.4f}, {df[col].max():.4f}]")

print(f"\nFeature dataset classes: {df['Class'].nunique()}")
print(f"BW image folders: {len(bw_species)}")
print(f"RGB image folders: {len(rgb_species)}")
print(f"BW matches feature dataset: {len(bw_species) == df['Class'].nunique()}")
```

```
# Image dataset quality checks
print("=" * 60)
print("IMAGE DATASET QUALITY CHECKS")
print("=" * 60)

bw_errors = 0
rgb_errors = 0
bw_dims = set()
rgb_dims = set()

for sp in bw_species:
    sp_path = os.path.join(BW_DIR, sp)
    for f in os.listdir(sp_path):
        try:
            img = Image.open(os.path.join(sp_path, f))
            bw_dims.add(img.size)
        except:
            bw_errors += 1

for sp in rgb_species:
    sp_path = os.path.join(RGB_DIR, sp)
    for f in os.listdir(sp_path):
        try:
            img = Image.open(os.path.join(sp_path, f))
            rgb_dims.add(img.size)
        except:
            rgb_errors += 1

print(f"BW corrupted files: {bw_errors}")
print(f"RGB corrupted files: {rgb_errors}")
print(f"BW dimensions: {bw_dims} (consistent: {len(bw_dims) == 1})")
print(f"RGB dimensions: {rgb_dims} (consistent: {len(rgb_dims) == 1})")

# Verify BW folder counts match feature data
match_count = 0
for sp in bw_species:
    class_num = int(sp.split('.')[0].strip())
    img_cnt = len(os.listdir(os.path.join(BW_DIR, sp)))
    feat_cnt = len(df[df['Class'] == class_num])
    if img_cnt == feat_cnt:
        match_count += 1

print(f"\nBW folder counts match feature data: {match_count}/{len(bw_species)}")
print("\nConclusion: Dataset is clean with no significant quality issues.")
```

SECTION 4

Classification

Naïve Bayes | SVM | MLP | Stacking | CNN Features

Presenter: Hye Ran Yoo

4.1 Classification – Data Preparation

Presenter: Hye Ran Yoo

Select & Clean Data

Feature Selection:

- **Target:** Species class label (Column 1)
- Excluded: Specimen Number (Column 2) – just an identifier
- Retained: All 14 numerical features
- Image dataset: Only 30 species present in BW & feature data

Data Cleaning:

- Non-consecutive class labels (1–15, 22–36) encoded to 0–29 using **LabelEncoder**
- No missing values, duplicates, or erroneous entries

```
# Encode class labels
le_cls = LabelEncoder()
y_cls_enc = le_cls.fit_transform(y_cls)
```

4.1 Classification – Construct Data

Presenter: Hye Ran Yoo

Normalization, Standardization, PCA, LDA

6 Prepared Datasets:

1. Original (14 features, unscaled)
2. Normalized (min-max to [0,1])
3. Standardized (z-score)
4. PCA-reduced (6 components)
5. LDA-reduced (14 components)
6. CNN-extracted (256-dim vectors)

Split: 80/20 stratified Train: 272 | Test: 68

```
# Construct 5 datasets for classification

# 1. Original
print("1. Original:", X_cls.shape)

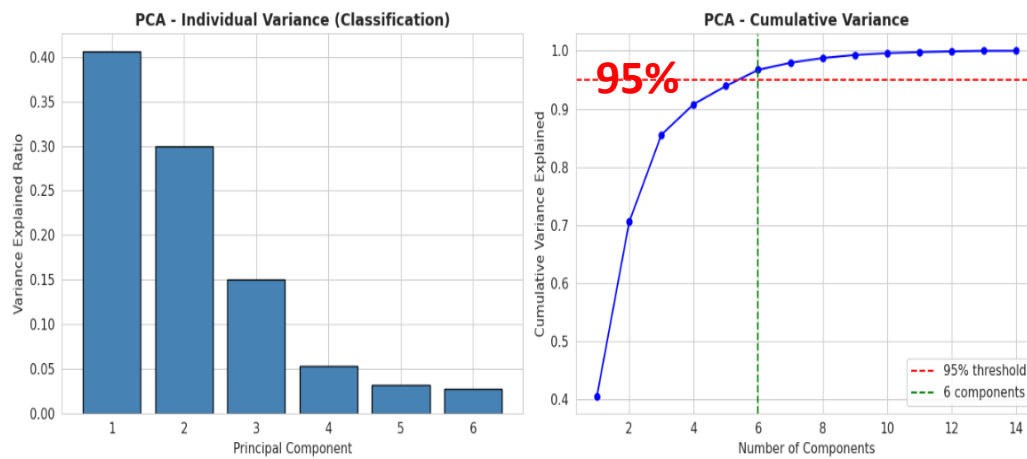
# 2. Normalized
mm_cls = MinMaxScaler()
X_cls_norm = pd.DataFrame(mm_cls.fit_transform(X_cls), columns=feature_cols)
print(f"2. Normalized: {X_cls_norm.shape}, range [{X_cls_norm.min().min():.3f}, {X_cls_norm.max().max():.3f}]")

# 3. Standardized
ss_cls = StandardScaler()
X_cls_std = pd.DataFrame(ss_cls.fit_transform(X_cls), columns=feature_cols)
print(f"3. Standardized: {X_cls_std.shape}, mean~{X_cls_std.mean().mean():.4f}, std~{X_cls_std.std().mean():.4f}")

# 4. PCA-reduced
pca_cls = PCA(n_components=0.95)
X_cls_pca = pca_cls.fit_transform(X_cls_std)
print(f"4. PCA-reduced: {X_cls_pca.shape}, variance explained: {pca_cls.explained_variance_ratio_.sum():.4f}")

# 5. LDA-reduced
lda_cls = LDA()
X_cls_lda = lda_cls.fit_transform(X_cls_std, y_cls_enc)
print(f"5. LDA-reduced: {X_cls_lda.shape}")
```

```
1. Original: (340, 14)
2. Normalized: (340, 14), range [0.000, 1.000]
3. Standardized: (340, 14), mean~0.0000, std~1.0015
4. PCA-reduced: (340, 6), variance explained: 0.9673
5. LDA-reduced: (340, 14)
```



4.1 Classification – CNN Feature Extraction

Presenter: Hye Ran Yoo

PyTorch LeafCNN Model

CNN Architecture:

- 3 Conv blocks (Conv2d → MaxPool → Dropout)
- Input: 128×128 RGB images
- Output: 256-dim feature vector
- Total parameters: 8,489,822

Purpose:

- Extract image features for traditional ML classifiers (NB, SVM, MLP)
- Also used as full CNN classifier

Limitation:

- Only ~11 images per class
- Insufficient for training from scratch

```
# CNN model for feature extraction (PyTorch)
class LeafCNN(nn.Module):
    def __init__(self, num_classes):
        super(LeafCNN, self).__init__()
        # Convolution Block 1
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1) # Shape: 128x128 -> 128x128 | Params: (3*3*3)*32 + 32 = 896
        self.pool1 = nn.MaxPool2d(2, 2) # Shape: 128x128 -> 64x64 | Params: 0
        self.drop1 = nn.Dropout2d(0.25)

        # Convolution Block 2
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1) # Shape: 64x64 -> 64x64 | Params: (3*3*32)*64 + 64 = 18,496
        self.pool2 = nn.MaxPool2d(2, 2) # Shape: 64x64 -> 32x32 | Params: 0
        self.drop2 = nn.Dropout2d(0.25)

        # Convolution Block 3
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1) # Shape: 32x32 -> 32x32 | Params: (3*3*64)*128 + 128 = 73,856
        self.pool3 = nn.MaxPool2d(2, 2) # Shape: 32x32 -> 16x16 | Params: 0
        self.drop3 = nn.Dropout2d(0.25)

        # Fully connected
        self.flatten = nn.Flatten() # Shape: 128x16x16 -> 32768 | Params: 0
        self.fc1 = nn.Linear(128 * 16 * 16, 256) # Shape: 32768 -> 256 | Params: (32768*256) + 256 = 8,388,864
        self.drop_fc = nn.Dropout(0.5)
        self.fc2 = nn.Linear(256, num_classes) # Shape: 256 -> num_classes(30) | Params: (256*30) + 30 = 7,710

    def forward(self, x):
        x = self.drop1(self.pool1(F.relu(self.conv1(x))))
        x = self.drop2(self.pool2(F.relu(self.conv2(x))))
        x = self.drop3(self.pool3(F.relu(self.conv3(x))))
        x = self.flatten(x)
        x = self.drop_fc(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x

    def extract_features(self, x):
        """Extract features from the fc1 layer (256-dim vector)."""
        x = self.drop1(self.pool1(F.relu(self.conv1(x))))
        x = self.drop2(self.pool2(F.relu(self.conv2(x))))
        x = self.drop3(self.pool3(F.relu(self.conv3(x))))
        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        return x

num_classes = len(np.unique(y_imgs_enc))
cnn_model = LeafCNN(num_classes).to(device)
```

4.1 Classification – CNN Feature Extraction

Presenter: Hye Ran Yoo

PyTorch LeafCNN Model

Key Steps of CNN Feature Extraction:

- **Batch Processing:**

Processed all 340 images in small batches (size=32) to prevent memory overload.

- **Hidden Layer Extraction:**

Stopped the CNN right before the final prediction to extract a **256-Dimensional Feature Vector**.

- **The Core Achievement:**

Upgraded our dataset from **14 human-measured features** → to **256** AI-discovered hidden visual patterns!

```
# Extract CNN features for all images
cnn_model.eval()

# Convert images to PyTorch tensors (N, C, H, W)
X_imgs_tensor = torch.FloatTensor(X_imgs).permute(0, 3, 1, 2).to(device)

# Extract features in batches
batch_size = 32
all_features = []
with torch.no_grad():
    for i in range(0, len(X_imgs_tensor), batch_size):
        batch = X_imgs_tensor[i:i+batch_size]
        feats = cnn_model.extract_features(batch)
        all_features.append(feats.cpu().numpy())

X_cnn_features = np.vstack(all_features)
print(f"CNN-extracted features: {X_cnn_features.shape}")
print(f"Each image -> {X_cnn_features.shape[1]}-dimensional feature vector")
```

```
CNN-extracted features: (340, 256)
Each image -> 256-dimensional feature vector
```

4.1 Classification – Integrate Data/Format Data

Presenter: Hye Ran Yoo

- **80/20 Split:**

All 6 dataset versions were identically divided into Train (272) and Test (68).

- **Fairness (stratify=y):**

Guaranteed that all 30 species are equally represented in both sets to prevent any bias.

- **Reproducibility (random_state=42):**

Locked the randomness to ensure consistent evaluation.

- **Code Efficiency ([:2]):**

Sliced the output to grab only the feature data (X_train, X_test) without uselessly copying the exact same answer labels (y).

```
# Format and split data for classification
X_tr_orig, X_te_orig, y_tr, y_te = train_test_split(
    X_cls, y_cls_enc, test_size=0.2, random_state=42, stratify=y_cls_enc)
X_tr_norm, X_te_norm = train_test_split(
    X_cls_norm, test_size=0.2, random_state=42, stratify=y_cls_enc)[:2]
X_tr_std, X_te_std = train_test_split(
    X_cls_std, test_size=0.2, random_state=42, stratify=y_cls_enc)[:2]
X_tr_pca, X_te_pca = train_test_split(
    X_cls_pca, test_size=0.2, random_state=42, stratify=y_cls_enc)[:2]
X_tr_lda, X_te_lda = train_test_split(
    X_cls_lda, test_size=0.2, random_state=42, stratify=y_cls_enc)[:2]
X_tr_cnn, X_te_cnn, y_tr_img, y_te_img = train_test_split(
    X_cnn_features, y_imgs_enc, test_size=0.2, random_state=42, stratify=y_imgs_enc)
```

```
=====
CLASSIFICATION - PREPARED DATASETS
=====
```

Original	Train: (272, 14)	Test: (68, 14)
Normalized	Train: (272, 14)	Test: (68, 14)
Standardized	Train: (272, 14)	Test: (68, 14)
PCA-reduced	Train: (272, 6)	Test: (68, 6)
LDA-reduced	Train: (272, 14)	Test: (68, 14)
CNN-extracted	Train: (272, 256)	Test: (68, 256)

```
Split: 80/20 stratified, classes in train: 30
```

4 Classification Models:

1. Gaussian Naïve Bayes

Probabilistic baseline; assumes
feature independence

2. SVM (RBF kernel)

Finds optimal hyperplane;
handles non-linear boundaries

3. MLP (128→64 hidden layers)

Learns complex non-linear mappings

4. Stacking Ensemble

NB + SVM + MLP → Logistic Regression
meta-learner with 5-fold CV

```
# Define classifiers
cls_models = {
    'GaussianNB': GaussianNB(),
    'SVM': SVC(kernel='rbf', random_state=42),
    'MLP': MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=500, random_state=42),
    'Stacking': StackingClassifier(
        estimators=[
            ('nb', GaussianNB()),
            ('svm', SVC(kernel='rbf', random_state=42)),
            ('mlp', MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=500, random_state=42))
        ],
        final_estimator=LogisticRegression(max_iter=1000, random_state=42),
        cv=5
    )
}

# Datasets (feature-based)
cls_datasets = {
    'Original': (X_tr_orig, X_te_orig, y_tr, y_te),
    'Normalized': (X_tr_norm, X_te_norm, y_tr, y_te),
    'Standardized': (X_tr_std, X_te_std, y_tr, y_te),
    'PCA': (X_tr_pca, X_te_pca, y_tr, y_te),
    'LDA': (X_tr_lda, X_te_lda, y_tr, y_te),
    'CNN-features': (X_tr_cnn, X_te_cnn, y_tr_img, y_te_img)
}
```

4.2 Classification – Training & Evaluation Code

Presenter: Hye Ran Yoo

Test Design:

- 6 datasets × 4 models = 24 experiments
- 80/20 stratified train-test split
 - Train: 272 instances
 - Test: 68 instances
 - All 30 classes represented
- Metrics (macro-average):
 - Accuracy
 - Precision
 - Recall
 - F1-Score
- Macro-average:
 - equal weight to all 30 classes regardless of size

```
cls_results = []

for ds_name, (Xtr, Xte, ytr, yte) in cls_datasets.items():
    for model_name, model in cls_models.items(): # NB, SVM, MLP, Stacking
        import copy
        clf = copy.deepcopy(model)
        clf.fit(Xtr, ytr)
        y_pred = clf.predict(Xte)
        rec = recall_score(yte, y_pred, average='macro', zero_division=0) # rec Recall
        acc = accuracy_score(yte, y_pred) # Accuracy
        prec = precision_score(yte, y_pred, average='macro', zero_division=0) # Precision
        f1 = f1_score(yte, y_pred, average='macro', zero_division=0)
        cls_results.append({
            'Dataset': ds_name,
            'Model': model_name,
            'Accuracy': round(acc, 4),
            'Precision': round(prec, 4),
            'Recall': round(rec, 4),
            'F1': round(f1, 4)
        })
    print(f" {ds_name:20s} | {model_name:12s} | Acc={acc:.4f} P={prec:.4f} R={rec:.4f} F1={f1:.4f}")

cls_results_df = pd.DataFrame(cls_results)
```

4.2 Classification – Results (Feature Data, PCA, LDA, CNN)

Presenter: Hye Ran Yoo

Dataset	Model	Accuracy	Precision	F1-Score
Original	GaussianNB	0.7941	0.8000	0.7867
Original	SVM	0.3529	0.2773	0.2724
Original	MLP	0.7353	0.7176	0.7094
Original	Stacking	0.5147	0.4770	0.4704
Normalized	GaussianNB	0.7941	0.8000	0.7867
Normalized	SVM	0.5147	0.4877	0.4740
Normalized	MLP	0.7353	0.7333	0.7171
Normalized	Stacking	0.6471	0.6122	0.6075
Standardized	GaussianNB	0.7941	0.8000	0.7867
Standardized	SVM	0.6765	0.6750	0.6567
Standardized	MLP	0.8088	0.8472	0.7979
Standardized	Stacking	0.6912	0.6889	0.6662

Dataset	Model	Accuracy	Precision	F1-Score
PCA	GaussianNB	0.6324	0.6067	0.6003
PCA	SVM	0.6176	0.6128	0.5951
PCA	MLP	0.7059	0.7278	0.6896
PCA	Stacking	0.6471	0.6833	0.6288
LDA	GaussianNB	0.7794	0.8022	0.7597
★ LDA	SVM	0.8382	0.8467	0.8313
LDA	MLP	0.8382	0.8661	0.8243
LDA	Stacking	0.7794	0.7917	0.7565
CNN-feat	GaussianNB	0.5000	0.4561	0.4473
CNN-feat	SVM	0.0882	0.0349	0.0363
CNN-feat	MLP	0.4265	0.3679	0.3710
CNN-feat	Stacking	0.4559	0.4611	0.4126

★ Best Classification Result: Dataset LDA Model SVM Accuracy 0.8382 Precision 0.8467 Recall 0.85 F1 0.8313

4.3 Classification – Evaluation & Key Findings

Presenter: Hye Ran Yoo

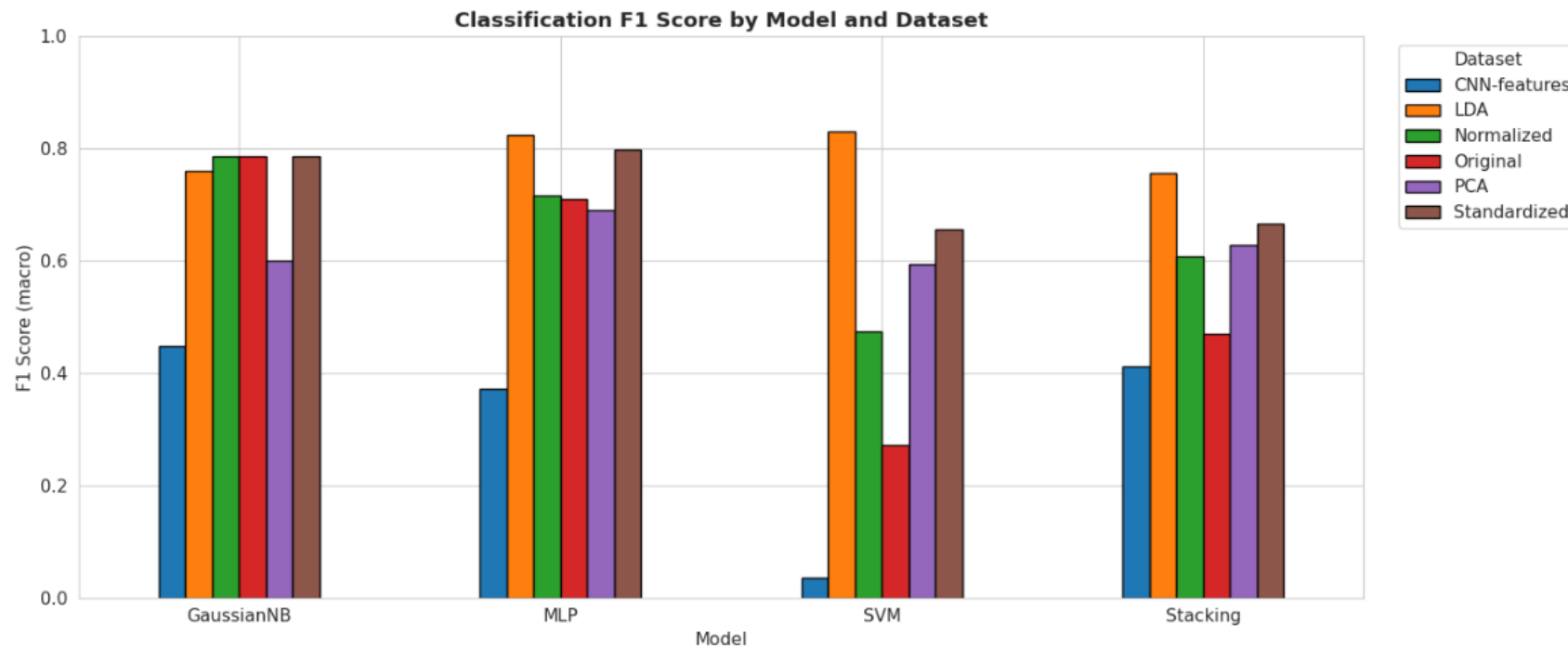
- **LDA is the most effective data preparation:**

LDA effectively maximized the distance between species, boosting accuracy for all models.

SVM accuracy: 35.29% (Original) → 83.82% (**LDA**) – **huge improvement!** (maximizes between-class variance)

- **CNN Limitations:** 340 images were insufficient for deep learning. The 14 CSV features were much more effective.

- **SVM vs. Stacking:** Complex ensembles (Stacking) overfitted the small dataset, while SVM maintained high accuracy.



SECTION 5

Clustering

K-Means | DBSCAN | Gaussian Mixture Model (EM)

Presenter: Peng Wang

5.1 Clustering – Data Preparation

Presenter: Peng Wang

Feature Selection:

- Class label & Specimen Number removed (unsupervised)
- True labels retained separately for evaluation only
- All 14 features initially included
 - Shape features: Eccentricity, Aspect Ratio, Elongation, ...
 - Texture features: Avg Intensity, Contrast, Smoothness, ...

Data Construction:

- Same 6 dataset versions: Original, Normalized, Standardized, PCA-reduced, LDA-reduced, CNN-extracted
- Standardization is critical for distance-based algorithms
- PCA expected to help by removing noisy dimensions

Key Difference from Classification:

- Full dataset used (340 instances) – no train-test split
- Labels only used for external validation (ARI, NMI)

5.2 Clustering – Model Selection & Code

Presenter: Peng Wang

```
clu_models = {
    'KMeans': KMeans(n_clusters=30,
                     random_state=42, n_init=10),
    'DBSCAN': DBSCAN(eps=2.0, min_samples=3),
    'GMM': GaussianMixture(n_components=30,
                           random_state=42)
}

for ds_name, X_data in clu_datasets.items():
    for model_name, model in clu_models.items():
        clust = copy.deepcopy(model)
        labels = clust.fit_predict(X_data)

        ari = adjusted_rand_score(y_clu_enc, labels)
        nmi = normalized_mutual_info_score(
            y_clu_enc, labels)
        sil = silhouette_score(X_data, labels)
```

3 Clustering Algorithms:

1. K-Means (k=30)
Partitional; assigns to nearest centroid iteratively until convergence
2. DBSCAN
Density-based; no predefined k
eps=2.0, min_samples=3
Can detect arbitrary-shaped clusters
3. Gaussian Mixture Model (EM)
Probabilistic; assumes data from mixture of 30 Gaussian distributions

Metrics: ARI, NMI, Silhouette Score

5.2 Clustering – Results

Presenter: Peng Wang

Dataset	Model	Clusters	ARI	NMI
Original	KMeans	30	0.2584	0.6238
Original	DBSCAN	3	0.0121	0.1997
Original	GMM	30	0.2925	0.6585
Normalized	KMeans	30	0.3807	0.7132
Normalized	DBSCAN	1	-1.000	-1.000
Normalized	GMM	30	0.3371	0.6931
Standardized	KMeans	30	0.3528	0.7104
Standardized	DBSCAN	3	0.0122	0.1903
Standardized	GMM	30	0.3558	0.6977
PCA	KMeans	30	0.3177	0.6791
PCA	DBSCAN	1	-1.000	-1.000
PCA	GMM	30	0.3320	0.6908
LDA	KMeans	30	0.4930	0.7830
LDA	DBSCAN	15	0.0318	0.3872
LDA	GMM	30	0.4553	0.7667
CNN-feat	KMeans	30	0.1794	0.5551
CNN-feat	DBSCAN	1	-1.000	-1.000
CNN-feat	GMM	30	0.1794	0.5551

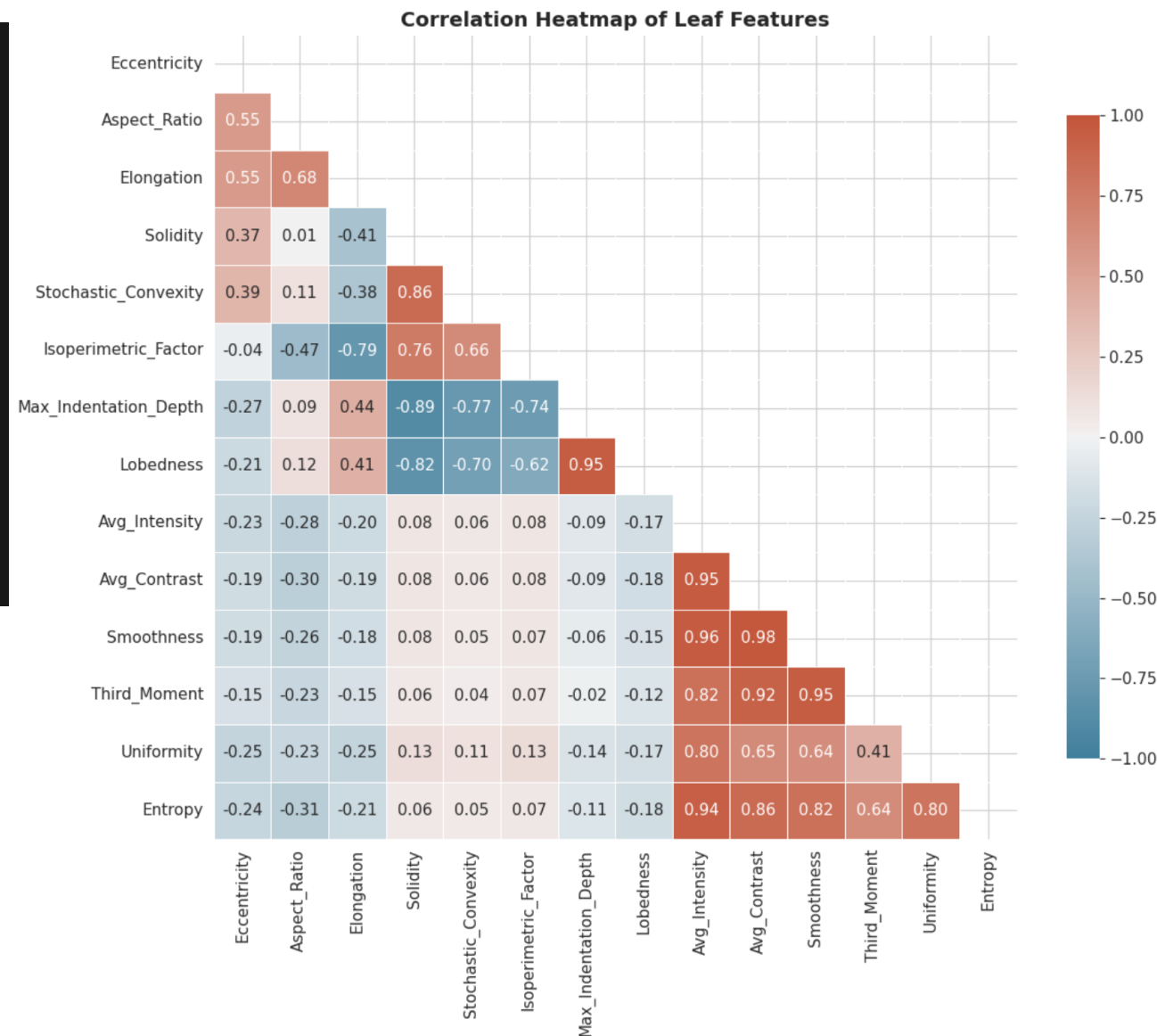
★ BEST: K-Means on LDA (ARI=0.493, NMI=0.783)

5.2 Clustering – Visualizations

Presenter: Peng Wang

```
# Correlation heatmap
fig, ax = plt.subplots(figsize=(12, 10))
corr_matrix = df[feature_cols].corr()
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, cmap=sns.diverging_palette(230, 20, as_cmap=True),
            annot=True, fmt='.2f', square=True, linewidths=0.5, ax=ax, vmin=-1, vmax=1,
            cbar_kws={'shrink': 0.8})
ax.set_title('Correlation Heatmap of Leaf Features', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print("Notable correlations (|r| > 0.5):")
for i in range(len(feature_cols)):
    for j in range(i+1, len(feature_cols)):
        r = corr_matrix.iloc[i, j]
        if abs(r) > 0.5:
            print(f" {feature_cols[i]} vs {feature_cols[j]}: r = {r:.3f}")
```



5.3 Clustering – Evaluation & Key Findings

Presenter: Peng Wang

1. LDA-reduced data produces the best clusters

- K-Means ARI: 0.258 (Original) → 0.493 (LDA)
- LDA projects into space that separates classes

2. DBSCAN struggled with high-dimensional data

- Collapsed all points into 1 cluster on Normalized, PCA, CNN
- Density-based approach couldn't find separation
- Better eps/min_samples tuning needed

3. K-Means and GMM performed comparably

- K-Means slightly better on most datasets
- Both benefited from standardization and LDA

4. Moderate ARI (<0.5) suggests 30 species are not easily separable using unsupervised methods alone

- High NMI (0.55–0.78) indicates partial agreement
- Expected given large # classes and small samples

SECTION 6

Outlier Detection

LOF | Isolation Forest | One-Class SVM

Presenter: Yinyuan Chen

6.1 Outlier Detection – Data Preparation

Presenter: Yinyuan Chen

Feature Selection:

- All 14 numerical features retained
- Class label & Specimen Number excluded
- Focus on high-variance / skewed features for anomaly detection:
 - Lobedness (skew=3.116), Aspect Ratio (3.325)
 - Uniformity (2.125), Solidity (-2.061)

Key Differences from Other Tasks:

- Potential outliers are NOT removed during cleaning
 - Detecting them is the objective!
- Full dataset (340 instances) used, no train-test split
- Contamination parameter set to ~10%

Standardization is critical: LOF and One-Class SVM

rely on distance measures; unscaled features would have disproportionate influence

```
out_models = {
    'LOF': LocalOutlierFactor(
        n_neighbors=20, contamination=0.1),
    'IsolationForest': IsolationForest(
        contamination=0.1, random_state=42),
    'OneClassSVM': OneClassSVM(
        nu=0.1, kernel='rbf', gamma='scale')
}

for ds_name, X_data in out_datasets.items():
    for model_name, model in out_models.items():
        det = copy.deepcopy(model)
        preds = det.fit_predict(X_data)
        # 1 = inlier, -1 = outlier
        n_outliers = (preds == -1).sum()
        pct = n_outliers / len(preds) * 100
```

3 Outlier Detection Algorithms:

1. Local Outlier Factor (LOF)

Density-based; compares local density to neighbors → sparse regions = outlier

2. Isolation Forest

Tree-based; anomalies need fewer splits to be isolated → efficient & scalable

3. One-Class SVM

Learns decision boundary around normal data; points outside = outlier

All use contamination $\approx$ 10%

6.2 Outlier Detection – Results

Presenter: Yinyuan Chen

Dataset	Model	Outliers	% Outliers
Original	LOF	34	10.00
Original	Isolation Forest	34	10.00
Original	One-Class SVM	32	9.41
Normalized	LOF	34	10.00
Normalized	Isolation Forest	34	10.00
Normalized	One-Class SVM	34	10.00
Standardized	LOF	34	10.00
Standardized	Isolation Forest	34	10.00
Standardized	One-Class SVM	36	10.59
PCA	LOF	34	10.00
PCA	Isolation Forest	34	10.00
PCA	One-Class SVM	34	10.00
LDA	LOF	34	10.00
LDA	Isolation Forest	34	10.00
LDA	One-Class SVM	38	11.18

Cross-Method Agreement

(Standardized Data)

ALL 3 methods: 15 outliers

At least 2: 31 outliers

ANY method: 58 outliers

→ 15 robust outliers represent
genuine anomalies confirmed
by all detection approaches

Key: One-Class SVM is most sensitive (38 outliers on LDA)

LOF & Isolation Forest are highly consistent (34 across most datasets)

6.3 Outlier Detection – Evaluation & Key Findings

Presenter: Yinyuan Chen

1. Consistent results across data preparations

- Most combinations: ~10% outliers (34/340)
- Confirms outlier detection is robust to scaling choices

2. One-Class SVM is the most sensitive detector

- Detected more outliers on LDA (38) and Standardized (36)
- May be overdetecting in some cases

3. Cross-method agreement strengthens confidence

- 15 instances flagged by ALL 3 methods → most reliable outliers
- These represent specimens with genuinely unusual morphology

4. Highly skewed features drive detection

- Aspect_Ratio (skew=3.325), Lobedness (3.116)
- Specimens with extreme values in these features flagged

Future Work:

- Examine botanical characteristics of identified outliers
- Test impact of outlier removal on classification & clustering
- Explore ensemble outlier detection methods

SECTION 7

Discussion of Results

Presenter: All Members

Data Preparation > Model Selection

- LDA consistently produced the BEST results across all tasks:
 - Classification: SVM accuracy 35% → 84% with LDA
 - Clustering: K-Means ARI 0.26 → 0.49 with LDA
- Standardization outperformed Original/Normalized for SVM and MLP (distance-based / gradient-based)
- PCA (unsupervised) showed mixed results:
 - 6 components explain 96.73% variance
 - But information loss hurt classification accuracy
 - Doesn't optimize for class separability
- Takeaway: Supervised dimensionality reduction (LDA) is highly beneficial when # classes is large relative to sample size

7. Discussion – CNN Features & Summary

Presenter: Yinyuan Chen

CNN-Based Features:

- Consistently underperformed across ALL tasks
- Only ~11 images per class – insufficient for CNN from scratch
- 256-dim CNN features did NOT beat 14 hand-crafted features
- Solution: Transfer learning (ResNet/VGG pre-trained on ImageNet)

Best Results Summary

Task	Best Dataset	Best Model	Key Metric
Classification	LDA	SVM / MLP	Accuracy = 83.82%
Clustering	LDA	K-Means	ARI = 0.493, NMI = 0.783
Outlier Detection	Standardized	All 3 (agreement)	15 robust outliers

Central Finding: Data preparation strategy has a GREATER impact on performance than model selection. The CRISP-DM methodology correctly emphasizes Data Preparation as a critical phase.

Project Summary:

- Applied CRISP-DM to UCI Leaf dataset (340 specimens, 30 species)
- Addressed 3 ML tasks: Classification, Clustering, Outlier Detection
- Systematically compared 6 data preparations × multiple algorithms

Classification:

- SVM on LDA: 83.82% accuracy, 84.67% precision, 83.13% F1
- MLP on LDA achieved identical accuracy

Clustering:

- K-Means on LDA: ARI = 0.493, NMI = 0.783
- Moderate agreement expected given 30 classes, small samples

Outlier Detection:

- 15 robust outliers identified by all 3 methods
- Consistent ~10% detection rate across preparations

Key Insight:

- Data preparation (especially LDA) > model selection
- CNN features need more data; recommend transfer learning

References

- Silva, P. & Marçal, A. (2013). Leaf [Dataset]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C53C78>
- Chapman, P., Clinton, J., Kerber, R., et al. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011).
Scikit-learn: Machine learning in Python. JMLR, 12, 2825–2830.
- Breunig, M. M., Kriegel, H. P., Ng, R. T., & Sander, J. (2000).
LOF: Identifying density-based local outliers. ACM SIGMOD, 29(2), 93–104.
- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest.
IEEE ICDM, 413–422.

Questions & Answers

Thank you for your attention!

Hye Ran Yoo
041145212

Peng Wang
041107730

Yinyuan Chen
041146649